



Lars

Look at chat log dinf.txt. It produced some artifacts [D-infinity-axioms.md](#), [lean4-core-logic.md](#) and LRSOD-in-CIC.lean. The goal is to make a .MD file which has the axioms in logic, set theory and point set topology and beyond needed to express Scott domains Dinf, then express the system of logic in Lean 4, and then show how to translate the axioms needed to express Dinf in Lean 4. My chat with Claude was a bit frustrating but almost got to the right point and then I ran out of tokens. Please finish the job in a fashion which you guess will be pleasing to me.

Composer 2.5 Fast

Reviewing the chat log and artifacts to produce the unified axioms-and-Lean translation document.

Verifying the Lean file compiles, then creating the unified document.

Creating the unified document: three parts plus a translation layer and references.

The unified document is at **LRSOD-in-CIC.md** . It follows the sequence you asked for in dinf.txt : LRSOD $\rightarrow$ CIC $\rightarrow$ LRSOD-in-CIC, with digressions removed and references added.

Structure

1. Part I — L, R, S, O, D

The 20 axioms/schemas + 2 rules, with the topology-first presentation (τ primitive, $\sqsubseteq$ defined), the L+R+S equivalence with the order-first formulation, and what is derived afterward (tower, inverse limit, $D_\infty \cong$

$$D_\infty \rightarrow {}_c D_\infty$$

) without new axioms.

2. Part II — CIC

Lean 4's kernel: universes, terms, judgments, K1–K7, E1–E10, inductive and quotient schemas, AX1–AX3, and a short interpretability note (CIC hosts a model of LRSOD; it is stronger than capped S).

3. Part III — LRSOD in CIC

Deep-embedding strategy, layer-by-layer correspondence tables, key Lean excerpts, the master dictionary, what remains to prove (construction of D_∞), and a constructivity

comparison vs Mathlib.

Companion artifact

`LRS0D-in-CIC.lean` had a syntax error on `∃!` (Lean core without Mathlib); it is fixed to explicit unique existence and compiles with `lean LRS0D-in-CIC.lean`.

The three earlier files (`D-infinity-axioms.md`, `lean4-core-logic.md`, `LRS0D-in-CIC.lean`) remain as source artifacts; `LRS0D-in-CIC.md` is the single paper that pulls them together. If you want, I can add a one-line pointer at the top of each older file directing readers to the unified doc, or wire the Lean file into the Lake project.

Lars

Can you bring up `LRS0D-in-CIC.lean` in Lean 4

Composer 2.5 Fast

Opening `LRS0D-in-CIC.lean` in the editor.

`LRS0D-in-CIC.lean` should now be open in Cursor.

This repo already has a `lean-toolchain`, so the Lean 4 extension should attach. If you don't see goals or diagnostics:

1. Confirm the **Lean 4** extension is enabled.
2. Run “**Lean 4: Restart File**” from the command palette on that tab.
3. Check the bottom status bar for a Lean version / “processing” indicator.

The file is standalone (not in `lake build ScottModels`), but it still compiles:

```
lean LRS0D-in-CIC.lean
```

If you want it fully integrated into the Lake project (LSP + `lake build`), I can wire it into `lakefile.toml`.

Lars

This seems build on scott1982, I want this particular presentation to be standalone, it's a thought exercise:

```
/home/catskills/.elan/toolchains/leanprover--lean4--v4.30.0/bin/lake setup-file /home/cat.  
failed:
```

```
stderr: error: scott1982: package directory not found: /home/catskills/Desktop/  
scott_models/./scott1982 Failed to configure the Lake workspace. Please restart the server after  
fixing the error above.
```

Composer 2.5 Fast

Making `LRS0D-in-CIC.lean` a minimal nested Lake package so the Lean server won't pull in the ScottModels workspace dependencies.

Creating a nested Lake package with no dependencies and configuring the Lean extension to treat it separately from ScottModels.

The problem was that `LRS0D-in-CIC.lean` lived at the **repo root**, so the Lean server picked up the main `scott_models lakefile.toml` and tried to load `scott1982`.

It's now a **standalone nested Lake package** with no dependencies:

```
LRS0D-in-CIC/  
  lakefile.toml      # no Mathlib, no scott1972/1980/1982  
  lean-toolchain  
  LRS0DInCIC.lean    # the embedding (Lake needs a valid module name)  
  LRS0D-in-CIC.md  
  README.md
```

`lake build` succeeds in that folder. The root copy of the `.lean` file is removed.

What you should do in Cursor:

1. Close the old tab if it still points at `scott_models/LRS0D-in-CIC.lean`.
2. Open `LRS0D-in-CIC/LRS0DInCIC.lean`.
3. Run “**Lean 4: Restart File**” (or restart the Lean server).

`.vscode/settings.json` now lists both `"."` and `"LRS0D-in-CIC"` as Lean project roots so this package and `ScottModels` can coexist.

Note: The Lean file is named `LRS0DInCIC.lean` (not `LRS0D-in-CIC.lean`) because hyphens aren't valid in Lean module names. The paper and folder still use the `LRS0D-in-CIC` name.

Lars

Make all the file names be free of -

Composer 2.5 Fast

Renaming files and folders to remove hyphens, and updating all references.

All hyphens are removed from paths and filenames:

```
LRS0DInCIC/  
  LRS0DInCIC.lean  
  LRS0DInCIC.md  
  README.md  
  lakefile.toml  
  lean-toolchain
```

`.vscode/settings.json` now points at `LRS0DInCIC`. `lake build` still succeeds.

Reopen `LRS0DInCIC/LRS0DInCIC.lean` (the old `LRS0D-in-CIC/` path is gone) and run **Lean 4: Restart File** if the server was still on the previous location.

Lars

Would Mathlib use "structure" in this case or something like "class"?

Composer 2.5 Fast

Good question — and the answer differs by layer. Let me check the actual Mathlib definitions rather than going from memory.

The user is asking about Mathlib conventions for whether certain axiom bundles like `TopologicalSpace` should be defined as `structure` or `class`. I'm recalling that `TopologicalSpace` is actually implemented as a `class` in Mathlib, and I'm starting to outline what that definition looks like with its core fields like `IsOpen` and the required axioms.

Now I'm explaining the reasoning: `TopologicalSpace` is a `class` because it represents canonical structure on a type—each type has at most one topology in a given context, and you want instance resolution to automatically find and apply it so that notation and lemmas work without explicit passing. For set theory axiomatizations, Mathlib doesn't use a class but instead constructs concrete models like `ZFSet`. The key design principle is to use `class` when there's one canonical instance per type that should be inferred, and `structure` for bundled mathematical objects where you might have multiple instances on the same carrier or need to quantify over them explicitly. Mathlib actually splits these into separate mixin classes rather than bundling them together — like `T0Space`, `QuasiSober`, and `SoberSpace`, each as their own `class` that layers on top of the base `TopologicalSpace`. I should check if the `scott_models` repo has Mathlib available, maybe through a `.lake packages` directory. Mathlib provides useful abstractions like `OrderBot` for bottom elements, `OmegaCompletePartialOrder` for domain-theoretic structures, and `ScottContinuous` for functions that respect the Scott topology. The key pattern in Mathlib is using Prop-valued classes to express axioms on existing types, and existential statements rather than explicit data fields for things like bases.

There's an interesting tension here: Mathlib embraces classical choice and uses existentials freely, while the standalone file avoids choice and keeps the basis as concrete data instead. That's a meaningful design choice worth noting. Mathlib also splits definitions between `structure` and `class` forms—like how `TopologicalSpace.IsTopologicalBasis` is defined—and uses `extends` to compose classes with data fields.

I should verify the actual Mathlib definitions for `QuasiSober` and `SecondCountableTopology` to be precise about whether `TopologicalSpace` is a class and how these pieces fit together. Let me check the source in the workspace to ground this in what's actually there....

Mathlib would use **both**, and the deciding question is whether the carrier is a *parameter* or a *field*.

The discriminator

Carrier as a parameter, one canonical instance per type, filled in by instance search → `class` .

Carrier as a field, many coexisting instances you quantify over → `structure` .

Your three layers land on different sides of that line:

Your layer	Mathlib form	Precedent
<code>SetTheory</code> (carrier <code>V</code> is a field)	<code>structure</code>	<code>Theory.ModelType</code>
<code>TopologicalSpace D</code>	<code>class</code>	it <i>is</i> a class in Mathlib
<code>DInfinityFoundations D</code>	<code>Prop</code> -valued <code>class</code> mixins	<code>SpectralSpace</code>

O is literally a class in Mathlib

```
class TopologicalSpace (X : Type u) where
  /-- A predicate saying that a set is an open set. Use `IsOpen` in the root namespace in
  protected IsOpen : Set X → Prop
```

S stays a structure

Because the carrier is a field, not a parameter. Mathlib's bundled models of a first-order theory do exactly this, for exactly your reason — you want to talk about several models at once:

```

structure ModelType where
  /-- The underlying type for the models -/
  Carrier : Type w
  [struc : L.Structure Carrier]
  [is_model : T.Model Carrier]
  [nonempty' : Nonempty Carrier]

```

Filter α is a structure for the same reason: many filters per type, so nothing should be inferred.

D would be split into Prop mixins, not one bundle

This is the biggest stylistic gap. Mathlib would not write `extends TopologicalSpace D` with the topology as a field; it would take `[TopologicalSpace D]` as a parameter and assert each condition as a separate `Prop` class. Your `D1` and `D2` already exist verbatim, and the split matches yours (`sober = T0 + quasi-sober`):

```

class T0Space (X : Type u) [TopologicalSpace X] : Prop where
  /-- Two inseparable points in a T0 space are equal. -/
  t0 : ∀ {x y : X}, Inseparable x y → x = y

```

```

class QuasiSober (α : Type*) [TopologicalSpace α] : Prop where
  sober : ∀ {S : Set α}, IsIrreducible S → IsClosed S → ∃ x, IsGenericPoint x S

```

Your `D3` is Mathlib's `PrespectralSpace` (compact-open basis) plus `SecondCountableTopology` (countability):

```

class PrespectralSpace (X : Type*) [TopologicalSpace X] : Prop where
  isTopologicalBasis : IsTopologicalBasis { U : Set X | IsOpen U ∧ IsCompact U }

```

And the aggregate shape a Mathlib-style `DInfinityFoundations` would take is exactly `SpectralSpace`:

```

class SpectralSpace (X : Type*) [TopologicalSpace X] : Prop extends
  T0Space X, CompactSpace X, QuasiSober X, QuasiSeparatedSpace X, PrespectralSpace X

```

Why your `structure` choice is nevertheless right here

Two reasons, and the second is the interesting one.

First, `class` with a bundled `TopologicalSpace` field would be a genuine diamond hazard: `D` could carry two topologies, one from `[TopologicalSpace D]` and one inside `DInfinityFoundations`, with instance search free to pick the wrong one. As a `structure` nothing is inferred, so the hazard doesn't arise.

Second, and this cuts to your constructivity discipline: notice that Mathlib's mixins are all `Prop`-valued, with existentials where you have data. Your `basis : Nat → (D → Prop)` is data; Mathlib would write `∃ b, b.Countable ∧ IsTopologicalBasis b`. Mathlib can afford that because it recovers the witness with choice — `QuasiSober` pays exactly that price:

```
noncomputable def IsIrreducible.genericPoint [QuasiSober α] {S : Set α} (hS : IsIrreducible α) :  
  α :=  
  (QuasiSober.sober hS.closure isClosed_closure).choose
```

Your file forbids `Classical.choice`, so keeping the basis as data is forced, and data in a class is only good style when the data is uniquely determined by the rest of the structure — the way `ωSup` is determined by the order in `OmegaCompletePartialOrder`. A countable basis is *not* uniquely determined, so it can't be a canonical instance. `structure` is the honest home for it.

Want me to add this as a short "Mathlib idiom vs deep embedding" subsection to `LRS0DInCIC.md`? It documents the `Prop`-plus-choice versus data-plus-choice-free trade-off, which is the same tension your `ScottModels` axiom audits are tracking.

The key insight is whether the carrier is a parameter or a field—if it's a field, Mathlib uses a structure, as seen in precedents like bundled models where you need to work with multiple instances simultaneously.

For the topological space, Mathlib makes it a class with the carrier as a parameter, allowing instance resolution to supply the canonical topology per type. For the D-infinity axioms, rather than bundling everything into one structure extending `TopologicalSpace`, Mathlib would split them into separate `Prop`-valued class mixins: `T0Space` for D1, `QuasiSober` for D2 (which aligns perfectly with the `sober = T0 + QuasiSober` decomposition), and `PrespectralSpace` for D3 with a basis

condition on compact open sets.

Mathlib actually has the exact aggregate pattern already in `SpectralSpace`, which extends multiple mixins like `T0Space`, `CompactSpace`, and `QuasiSober` over a `TopologicalSpace` instance parameter rather than bundling it as a field. This reveals a key tension: their file treats the basis as concrete data (a function from naturals to propositions), whereas Mathlib's approach would make it a Prop-valued property instead.

The trade-off is significant — Mathlib's Prop-form avoids instance resolution issues and diamond problems, but requires noncomputable definitions and classical choice to extract the basis back out. Their choice-free constructive stance is precisely why keeping the basis as data makes sense for this exercise, even though it diverges from Mathlib's style.

Using `extends` with data in a structure is fine since nothing gets inferred, but as a class it would risk carrying multiple topologies and confusing instance search. Interestingly, Mathlib does allow data fields in classes when the data is genuinely canonical and unique — like `OmegaCompletePartialOrder` bundling `ωSup` — so the constraint isn't absolute, just pragmatic.

For their exercise, `structure` is the right call: it's a deep embedding with no inference or choice involved. The Mathlib-idiomatic approach would flip O and D to classes, but that's not what they're doing here. I should offer to add a section explaining this to the markdown.

I'll cite the existing code using absolute...

Turn 7

Lars

Yes add that. I have updated the file a little. This is a tutorial/thought exercise which is separate from scott_models "professional" goal.

Composer 2.5 Fast

Let me read your updated Lean file and the paper's current structure before adding the section.

The reference formatting change is noted. I'm planning to add a new subsection on the Mathlib

idiom versus deep embedding distinction using `structure` versus `class`, positioning it right after the Master correspondence section and before the Constructivity note, since that note already references the Mathlib route approach....

I need to insert a new section after "Master correspondence" that distinguishes between carriers as parameters versus fields, maps out the layer structure, notes that Mathlib uses `TopologicalSpace` as a class, keeps `S` as a structure following `ModelType` precedent, and explains how `D` would work as Prop-valued mixins—all without referencing the professional repo.

I should also sketch the Lean code showing the Mathlib-idiomatic approach for tutorial purposes, add a Mathlib reference entry, and include a brief scope note in the abstract clarifying this is a standalone tutorial exercise....

Mathlib provides `specializationPreorder` and `specializationOrder` (the latter as a definition requiring a local instance attribute), along with a `Specialization` type synonym that has the necessary instances. Their convention for the specialization order is flipped from mine—they define $x \leq y$ to mean $y \rightsquigarrow x$, so the direction of the specialization relation is reversed.

Let me work through what this means: in Mathlib, $x \rightsquigarrow y$ (written `Specializes x y`) means $\Diamond x \leq \Diamond y$, which is equivalent to $y \in \text{closure } \{x\}$. Using the LRSOD definition where $x \sqsubseteq y$ means every open set containing x also contains y , I can verify this matches—if every neighborhood of x contains y , then $\Diamond x \leq \Diamond y$ as filters. So LRSOD's $x \sqsubseteq y$ corresponds exactly to Mathlib's $x \rightsquigarrow y$, but Mathlib's preorder flips this by setting $x \leq y \leftrightarrow y \rightsquigarrow x$.

This is a key difference: Mathlib's specialization order is the *dual* of the domain-theoretic order, oriented toward algebraic geometry where generic points are maximal rather than minimal. So in Mathlib's convention, $x \leq y$ means y is a specialization of x , which is backwards from the Scott/domain reading where more information flows upward. So `OrderDual` or `Specialization  $\alpha^{\text{od}}$` is needed to flip the order back to domain-theoretic orientation. Let me verify...

Now adding the section. Since you pruned the `ScottModels` cross-references, I'll keep it self-contained and tutorial-framed.

I'm confirming the relationship between the LRSOD definition and Mathlib's specialization order: the condition that for all open sets U , if x is in U then y is in U is equivalent to the neighborhood filter ordering, which matches Mathlib's $x \rightsquigarrow y$ notation in the `specializationPreorder`.

The key insight is that Mathlib reverses this: their order relation $x \leq y$ in the specialization preorder actually means $y \leadsto x$, so the bottom element in LRSOD (which only has the whole space as an open neighborhood) becomes the top element in Mathlib's order, making it an `OrderTop`.

Now I'm planning the markdown section structure: I'll explain the distinction between Mathlib's class-based approach for `TopologicalSpace` versus the deep embedding using structures, then create a correspondence table showing how each LRSOD axiom maps to Mathlib's topological classes, and finally justify why this file maintains the structure approach rather than relying on inference....

I need to add a `Scope` section right after the abstract and include a Mathlib reference, then update the constructivity note to be more precise about the Prop-valued mixin classes like `T0Space`, `QuasiSober`, and `PrespectralSpace` instead of the current `ScottDomain`-style description. I should also add the mathlib4 reference to the bibliography.

The user's title uses LaTeX notation for domain theory, but there's a syntax error in the abstract with an unbalanced parenthesis and missing space. I'll fix this typo since it's a minor correction that improves clarity.

I should also add a **Scope** line to clarify that this is a tutorial exercise separate from the main `scott_models` work. For consistency with the rest of the document body, I'll keep my new section using plain D_∞ notation rather than switching to math mode, since the existing content mixes styles.

I'm noting a precision point about the axiom count: Mathlib's `TopologicalSpace` uses three fields (`isOpen_univ`, `isOpen_inter`, `isOpen_sUnion`), while the `O` definition has `O1` split into two separate fields for empty and full sets, but Mathlib derives the empty case from the union of an empty family, so the three axioms `O1`–`O3` align correctly with Mathlib's structure.

I should add a clarifying sentence that Mathlib derives `isOpen_empty` from `isOpen_sUnion` applied to the empty family, whereas the Layer `O` definition states `openEmpty` explicitly—a useful tutorial detail that shows the difference in presentation.

I'll also rebuild `LRSODInCIC.lean` to verify it still compiles after the user's edits, even though the content appears unchanged.

Added, and `lake build` is still green.

The new subsection “**Mathlib idiom vs deep embedding: `structure` or `class` ?**” sits between the master correspondence table and the constructivity note, covering:

- **The discriminator** — carrier as parameter (`class` , one canonical instance, found by inference) versus carrier as field (`structure` , many instances quantified over explicitly), with a table placing S on one side and O, D on the other.
- **O is already a class in Mathlib**, quoted, plus the detail that Mathlib states three fields and derives “`∅` is open” from `isOpen_sUnion` of the empty family, where Layer O asserts `openEmpty` explicitly.
- **S stays a structure**, with `Theory.ModelType` and `Filter α` as precedents.
- **D would be Prop mixins, not a bundle** — `T0Space` , `QuasiSober` , `PrespectralSpace` + `SecondCountableTopology` , the `SpectralSpace` aggregate pattern, and a sketch `DInfinitySpace` class in idiomatic form.
- **Why this document keeps `structure`** — the two-topology diamond hazard; the `Prop` -plus-choice versus data-plus-choice-free fork (`QuasiSober sober ... .choose` is `noncomputable` , and a countable basis isn't uniquely determined the way `ωSup` is, so it can't be a canonical instance); and an orientation warning, since `⊆` here is Mathlib's `Specializes` but `specializationPreorder` installs the reverse order, making D4's `⊥` the *top* under Mathlib's convention.

Two small edits alongside it: a **Scope** line under the abstract marking this as a self-contained tutorial exercise rather than production infrastructure, and a `[mathlib4]` reference entry listing the cited files. I also closed an unbalanced paren in the abstract left over from your retitling, and sharpened the constructivity table's Mathlib row from `ScottDomain` -style typeclasses to “ `Prop` mixin classes + `.choose` ”.

Turn 8

Lars

In the .MD file before you use “field” and “parameter” please define them for dummies because I can't tell the difference.

Composer 2.5 Fast

Let me find where those terms first appear in the document.

I need to locate where "field" and "parameter" are first introduced in the documentation so I can add plain-language definitions before they're used.

I'm identifying where to add a vocabulary note—right at the start of Part III before line 239 where "structure" and "field" first appear in context. I'll write a beginner-friendly explanation with a concrete side-by-side example showing what a structure is (a labeled bundle of data) and what fields are (the individual labeled slots within it).

Now I'm clarifying the distinction between parameters and fields: parameters appear in the declaration header before `where` and must be supplied upfront to even name the type, while fields are the slots inside the bundle itself. I'm illustrating this with two Lean examples—one where `D` is a parameter that must be fixed before you can reference the type, versus one where the carrier is a field that can vary between different instances. The key practical difference is that `TopologicalSpace Nat` is a complete type name, but `TopologicalSpace` alone isn't yet a type, whereas `SetTheory` stands on its own with the carrier bundled inside each value.

I'm also noting that this distinction matters for class instance search, which looks at parameters to find instances—you can't search for something whose carrier is hidden inside a field. That's why classes use parameters and structures use fields. Now I'm deciding where to place this vocabulary section in the document—it should go right after the "Part III" heading, before the Strategy paragraph, so that when Strategy mentions "fields" the term is already defined.

Adding a vocabulary subsection at the top of Part III, before the first use of either term.

I'm refining the section around line 331 to clarify that the key distinction isn't whether something is an axiom bundle, but rather whether the carrier functions as a parameter or a field. I'll add a back-reference to the vocabulary definitions established earlier in Part III and adjust the bullet points so they build naturally on those foundational concepts.

Added a **Vocabulary:** `structure`, `field`, `parameter` subsection at the head of Part III, ahead of the first use of either word.

It defines the three terms in plain language, then shows the distinction side by side using the document's own two declarations — `TopologicalSpace (D : Type) where ...`, where `D` sits in

the header, versus `SetTheory` where `V : Type`, where the carrier is just another slot. The point is made concrete by asking *when* the carrier gets chosen: `TopologicalSpace` isn't a type until you write `TopologicalSpace Nat`, whereas `SetTheory` is complete on its own, so two values of it can have unrelated carriers and you can write `∀ T : SetTheory, ...`.

The plain-English version is the box analogy: a parameter is printed on the outside of the box so you can ask for that box by name; a field is what's inside, invisible until you open it. That sets up why the distinction decides `structure` versus `class` — instance search can only look at what's on the outside, so it can find "the topology on `D`" but has nothing to search by when the carrier is hidden in a field.

The later section now says "in the sense fixed at the start of this part" so the two passages read as one thread.

Turn 9

Lars

add "class" in the vocab section

Composer 2.5 Fast

I need to add `class` to the vocabulary section alongside `structure`, defining it as a structure that Lean automatically fills in through instance search, with instances declared using `instance` and requested with square brackets. I'll update the heading to include all four terms, add a definition paragraph with a code example showing the instance and bracket syntax, and place the `class` definition logically after `structure` but before discussing fields and parameters. Writing code examples... For classes, Lean automatically makes projections instance-implicit, so `TopologicalSpace.isOpen U` would resolve through instance search, and after opening the namespace I could write `isOpen U` directly. I'll keep this as a sketch and clean it up.

`class` is now defined in the vocabulary section alongside the other three terms.

The definition frames it as the same record type as `structure`, differing only in one permission: Lean is allowed to supply the bundle automatically, registered with `instance` and requested with

square brackets rather than by name. The distinction is who produces the value — you explicitly, or instance search silently.

A second code block makes that visible at the use site: with the `structure` as actually declared here you write `T.isOpen U` and must name `T`, whereas had it been a `class` you would write `isOpen U` with nothing to name. The block is labelled a sketch since this file's `TopologicalSpace` really is a `structure`. That leads directly into the existing paragraph on why instance search needs the carrier on the outside.

Turn 10

Lars

Can you think of a simple example for the bottom of the .lean file of `DInfinityFoundations` on some type. Ordinarily for class this would be instance but since we aren't using class this would be "example"?

Composer 2.5 Fast

Good question on terminology, and yes — `instance` is only for `class`. For a `structure` you'd use `def` (named, reusable) or `example` (anonymous, discards the value). I'll use `def` so it can be referenced and audited, and mention `example` in the comment.

Let me write the smallest non-degenerate witness: Sierpiński space, the two-point flat domain.

The user wants me to add a concrete example of `DInfinityFoundations` instantiated on some type at the end of the file. For the terminology question: since `DInfinityFoundations` is a structure rather than a class, I should use `def` to create a named instance (so it can be referenced later) rather than `instance`, which is reserved for classes.

Now I'm thinking through what the simplest type satisfying the axioms D1–D4 would be. The one-point space `Unit` seems like a natural candidate. If I set the carrier to `Unit` and declare everything open (`isOpen` always returns true), then the axioms should hold: the empty and full sets are trivially open, the intersection and union of open sets are open, and the specialization preorder becomes reflexive since `Unit` is a subsingleton—any two elements are equal by reflexivity.

For the sobriety axiom, I need to verify that every closed irreducible set corresponds to a unique point. With everything open, the closure of a singleton at any point `y` becomes the set of all `x` where `x` is specialized by `y`, which in this case means all elements of `Unit` since there's only one.

The key issue is that proving `C = clSingleton y` as an equality of functions requires function extensionality and propositional extensionality — both of which are available in Lean core and fall within the acceptable axioms for this construction.

Now I'm checking which predicates on `Unit` are irreducible and closed. The constant `False` predicate fails the irreducibility requirement since it has no witnesses. For `True`, I need to verify the closure conditions: that it can't be decomposed into a union of two proper closed subsets. If `C = True` and I assume some decomposition exists with predicates `C1` and `C2`, I need to derive a contradiction from the required properties.

The key insight is that from the decomposition conditions, both `C1 ()` and `C2 ()` must be false, but the closure requirement forces at least one to be true. This contradiction shows `True` is irreducible.

To prove sobriety, I take an irreducible closed predicate `C` and show it equals the closure of a singleton. Since `C` is irreducible, it has a witness, so `C ()` holds. Using function extensionality and propositional extensionality, I can establish that `C` and the singleton closure are equal everywhere.

For the D3 basis, I'm setting up the trivial basis where every open set is just `True`, then verifying the compactness condition: given any open cover of the whole space, I need to extract a finite subcover. Since we're covering the single point `()`, there's some open set `U i ()` in the cover, and I can use `Exists.elim` to pull out that index `i` within a proof of a Prop—no choice principle needed. Now I'm working through the basis generation property to show that for any open set `U` and point `x` in it, there exists a basis element containing `x` that stays within `U`.

For the basis intersection property, I'm checking that the intersection of any two basis elements can be expressed as another basis element—taking `k = 0` makes this trivial since `True ∧ True` equals `True`. This confirms that the `Unit` type with the all-open topology satisfies the required structure, and it turns out this is exactly the one-point domain D_0 , which is the foundational base case in Scott's hierarchy.

A more interesting example would be the flat two-point domain—Sierpiński space with `{⊥, τ}`

and opens $\{\emptyset, \{\tau\}, D\}$ —which is a genuine nontrivial ω -algebraic pointed domain that's T_0 but not T_1 and sober. I'm considering how to define this in Lean by setting $D := \text{Bool}$ with $\perp = \text{false}$ and $\top = \text{true}$, then capturing the upward-closure property of opens through a condition like: if the bottom element is in the open set, then everything must be in it. So the open set $U = (\cdot = \text{true})$ fails for leq true false because $U \text{ true}$ holds but $U \text{ false}$ doesn't, confirming the order is correct. For the T_0 separation axiom, I need to show that if two elements satisfy the same open sets, they're equal — this requires case analysis on Bool , which is manageable. For the bottom element, false works as the least element since any open set U that contains false must also contain true by the openness condition.

Now I'm setting up a countable compact-open basis closed under finite intersections. The up-sets $\{\emptyset, \{\text{true}\}, D\}$ form the topology, and I need to verify that any two basis elements intersect to give another basis element. The intersections all check out: D with anything gives that thing, and $\{\text{true}\}$ with itself is $\{\text{true}\}$. I'm encoding this basis as a function from natural numbers to predicates on Bool .

For compactness, I need to show that D and $\{\text{true}\}$ are compact by extracting finite subcovers from arbitrary covers—for D this means finding indices for both false and true , then constructing a finite cover using $\text{Fin } 2$. This requires case analysis but is manageable.

Now moving to sobriety: closed irreducible sets are the complements of up-sets. The closed sets in D are $\emptyset$, $\{\text{false}\}$, and D itself. $\{\text{false}\}$ is irreducible with generic point false , and D is irreducible with generic point true since its closure equals D . D can't be decomposed into two proper closed subsets, confirming it's irreducible.

To prove the space is sober, I need to show every irreducible closed set has a unique generic point. Since we're working with Bool , I can use the law of excluded middle (which is already an axiom in the file) to case on whether true is in an irreducible closed set C . If C contains true , then by closure properties, C must also contain false , making C equal to D . If C doesn't contain true , then C must be $\{\text{false}\}$ since C is nonempty.

For uniqueness, I need to verify that the singleton sets are actually distinct as functions. The Sierpiński space example would be more instructive but requires substantial proof work—probably 60-100 lines of fiddly Lean code with real debugging risk. Since the user asked for something simple, I should present both approaches: keep Unit as the straightforward example but acknowledge the Sierpiński space as the more interesting case. `def` is the right choice here since I want to name and reuse the instance, and I can mention `example` as an alternative in a

comment for the user's reference.

I should also add a `#print axioms` audit at the end to show what axioms are being used—`propext`, `Quot.sound` (which gives us `funext`), and potentially `lem` if needed. For the Unit example specifically, the sober property will require `propext` and `Quot.sound`, which is a nice demonstration of their constructivity concerns.

Now I'm writing out the Unit example code... Defining the structure instance... I'll use tactic proofs for robustness instead. For the Unit case, I can pattern match directly with

```
match x, y with | (), () => rfl
```

 to handle the trivial structure. Now I'm working on the `sober` property, starting with introducing the closed set and its irreducibility assumption, then extracting the witness from the hypothesis.

Finishing the proof by constructing the singleton closure... Unfolding `clSingleton () y` with our trivial topology gives $\forall U, \text{True} \rightarrow U () \rightarrow U ()$, which is provable by the identity function. So I can finish with `propext` and the two directions of the equivalence, using `cases x` to handle the unit case. For uniqueness, I need to show any `y'` satisfying the condition equals `()`, which follows by case analysis on the unit type.

Now for the basis definition: I'm setting up a trivial basis where every element maps to the constantly-true predicate, with the basis being open by triviality. For compactness, I need to show the whole space is compact—given any open cover of the unit type, I can extract a single covering set and use that as my finite subcover. For the unit case, I need to handle the basis generator by introducing the neighborhood and element, then using case analysis to reduce everything to the trivial unit type. The basis cap operation just returns a trivial neighborhood. For the bottom element, I'm setting it to the unit value with an axiom that handles any neighborhood by case analysis. Once I get the unit domain compiling, I'll move on to the more involved Sierpiński space construction.

Actually, the unit domain is too degenerate—every axiom holds vacuously. Sierpiński space is the real canonical example: it's the two-point flat domain with a proper order relation, non- T_1 separation, and a genuine compact-open basis. That's what will actually illuminate the structure, so I should focus my effort there and maybe just sketch the unit case.

Now I'm verifying that Sierpiński satisfies the topology axioms: the openness condition is that if false is in the set, then true must be too. Union of opens works because if any component contains false, I can use that same component to witness true. Intersection and the empty/full sets follow

straightforwardly from the implication structure.

I'm defining the order relation using opens: $x \leq y$ when every open set containing x also contains y . To prove the T0 separation axiom, I need to show that if two points are indistinguishable by opens, they're equal. For the test set $U := \{\text{true}\}$, I need to verify it's open—that $\text{false} = \text{true}$ implies $\text{true} = \text{true}$, which holds trivially by reflexivity.

Then I apply the hypothesis to this set, getting an equivalence between $x = \text{true}$ and $y = \text{true}$. Checking each case: when both are false or both are true, reflexivity works. When x is false and y is true, the equivalence gives me $\text{false} = \text{true}$ from the forward direction, which contradicts `Bool.noConfusion`. When x is true and y is false, the reverse direction gives $\text{false} = \text{true}$, also a contradiction. So this case analysis completes the proof.

Now for the bot element: I set it to false with an axiom stating that any open set containing false must contain all points. This works because if U contains false, then for any x , either x is false (use the hypothesis directly) or x is true (apply the axiom). Finally, I'm starting to define the basis using a function that takes an index.

For $n=0$, the basis element is the whole space (always true). For $n+1$, the basis element is the singleton $\{\text{true}\}$. The openness proof is straightforward: the whole space is trivially open, and the singleton $\{\text{true}\}$ is open because the condition " $y = \text{true}$ " implies " $y = \text{true}$ " by reflexivity. Both basis elements are compact, and the compactness argument uses a finite subcover indexed by `Fin 2` with decidable equality.

For the covering property, when $x = \text{false}$, I need to show there exists j such that x is in $U(f(j))$. The challenge is reducing the conditional expression `if j = 0 then i0 else i1` definitionally. Using pattern matching on `Fin 2` or working with the underlying natural number representation with decidable equality on naturals should make the reduction work cleanly—when $j.\text{val} = 0$, the condition `0 = 0` reduces to true by reflexivity, so the conditional evaluates to i_0 without needing external simplification tactics.

Actually, there's a much cleaner approach: since the space is compact, any open cover must contain an open set covering false, and since open sets are upward-closed, that same set contains true. So a single index suffices—I can define f as the constant function returning that index, and the covering property follows immediately from the openness of that single set. $\text{is 1 true} = (\text{true} = \text{true}) \checkmark; \forall y, y = \text{true} \rightarrow \bigcup y: \text{subst} \rightarrow h x \checkmark$.

- `basisCap`: $\forall m n, \exists k, \forall x, (\text{basis } m \ x \wedge \text{basis } n \ x) \leftrightarrow \text{basis } k \ x$.

- $m=0, n=0: k=0: (\text{True} \wedge \text{True}) \leftrightarrow \text{True} \checkmark$
- $m=0, n=\text{succ}: k=n(\text{succ}): (\text{True} \wedge b=\text{true}) \leftrightarrow (b=\text{true}) \checkmark$
- $m=\text{succ}, n=0: k=m \checkmark$
- $m=\text{succ}, n=\text{succ}: k=m \checkmark ((b=\text{true} \wedge b=\text{true}) \leftrightarrow b=\text{true})$

D2 sober: Given C closed (

meaning $C \text{ true} \rightarrow C \text{ false}$ by contraposition) and irreducible (so $\exists x, C x$), I need to find the unique generic point. Using the law of excluded middle on $C \text{ true}$, if $C \text{ true}$ holds then $C \text{ false}$ follows from the closed property, and if $C \text{ true}$ doesn't hold, I can derive a contradiction from irreducibility, so $C \text{ false}$ must be true by classical logic.

Now I'm checking whether C equals the closure of the singleton $\{\text{true}\}$. The closure of $\{\text{true}\}$ consists of all points x where every open set containing x also contains true . For false , this reduces to the openness condition, which holds, so false is in the closure. For true , it trivially satisfies the condition. By function and propositional extensionality, C is indeed the closure of $\{\text{true}\}$, and the unique generic point is true . For uniqueness, if some other point y' were the generic point with C equal to the closure of $\{y'\}$, then y' must be true —if y' were false , evaluating C at true would require every open set containing true to also contain false , which fails for the singleton open set $\{\text{true}\}$.

In the case where C doesn't hold at true , the witness from the existential must be false , giving $C = \text{closure of } \{\text{false}\}$. At false , the closure condition holds; at true , it fails since true is not below false in the order. Both sides of the equality are false at true and true at false , so by propositional extensionality they're equal.

For the Lean proof, I'm looking at roughly 80 lines with tactics to handle both the case analysis and the equality verification. I'll focus on Sierpiński as the main example since it's the smallest nontrivial space and genuinely simple, though there's some risk it might get fiddly — if so, I can fall back to Unit as a simpler alternative.

Actually, Sierpiński does satisfy all the D1–D4 axioms as a pointed ω -algebraic domain, which is a nice teaching moment: the axioms characterize the class of domains where the tower construction works, not D_∞ uniquely. Sierpiński is another model of those axioms, and reaching D_∞ specifically requires the full tower and inverse limit, not just D1–D4 alone.

Let me verify D3 more carefully — the basis needs to be closed under finite intersection and each element compact. Both $\{\text{true}\}$ and the whole space are compact, so that checks out. And for the basis generation property, if U is an arbitrary open containing false , then U must be an up-set

containing everything, so the basis element at 0 (the whole space) is contained in U as required.

I'm wondering if Unit might be worth including as an even simpler warm-up example.

Writing code examples... Defining the topology inline with the structure instance avoids needing a separate named definition, making unfolding definitional so tactics like `intro` and `exact` work directly without extra simplification steps.

I'll go with two named definitions for clarity: `sierpTop` for the topological space on `Bool`, and `sierpDomain` for the domain structure that references it. Now I'm writing out the concrete implementation with the Sierpiński space as a worked example.

Writing the Lean code... Finishing the topology definition... Writing tactic proofs... When I instantiate the structure, `self.toTopologicalSpace` gets replaced with `sierpTop`, so the `isOpen` in the field type becomes `sierpTop.isOpen`. For the false/true case, I need to show that `Bool.noConfusion` can derive a contradiction from `false = true`, which it should handle correctly. The `decide` tactic is available in core Lean. After running `cases` on both variables, I get four cases in the expected order: false/false, false/true, true/false, true/true. Now I'm setting up helper lemmas about `clSingleton` before the main definition. I need to prove that `sierpTop.leq false y` holds for all `y` by showing that if `U false` is true for some open set `U`, then `U y` must also be true—which splits into the false case (trivial) and true case (using the openness property).

Writing sober proof... Continuing the proof cases...

The issue with using `hy' ▸` directly for function equality is getting messy, so I'm switching to extracting the pointwise equality instead. I'll apply `congrFun` to get `C true = clSingleton y' true`, then use `Eq.mp` or the implication direction to handle the propositional equality cleanly.

For the two cases: when `y' = false`, I cast the equality to derive a contradiction with `leq_true_false`; when `y' = true`, I cast the symmetric equality to get `C true` from `leq_true true`, then contradict it with the negation assumption.

Now I need to define some helper lemmas about the Sierpinski topology ordering — one showing any element is less than or equal to true, another for false being less than or equal to itself, and a third proving true is not less than or equal to false.

For the basis, I'm setting up a function indexed by natural numbers where zero maps everything to

true, and any successor maps to the predicate that checks if the element equals true. Then I need to verify these basis sets are open in the topology. Writing the basisCpt proof... Writing the basis generator... Writing basis case handlers... I need to decide whether to include the Sierpiński example alone or both it and Unit, noting that sobriety is where classical reasoning enters the picture. Since the file already has `lem` available, I can use it directly. I'm settling on just the Sierpiński example with a comment mentioning Unit works too, and I'll note that these finite domains exemplify the class of pointed ω -algebraic domains that D1–D4 axiomatize.

There's a type error on line 284 where `decide` can't handle the expected type because it contains a free variable `n†`. I need to simplify the match expression first before applying the tactic, either by using `simp only` to reduce it or by constructing an explicit proof using `Bool.noConfusion`.

Since `hx` has type `false = true` after reduction, I can use `Bool.noConfusion hx` directly — the unification should handle the match reduction through weak head normal form. I'll apply the same fix to the `basisGen` branch where `hy : basis 1 false` appears, and also remove the `sorryAx` from the axiom list that was introduced by the error, then rebuild.

Green, and the audit reads `[lem, propext, Quot.sound]` as expected. Let me verify my claim that the other layers are choice-free.

The build succeeded with no errors, and the audit confirms exactly the dependencies I expected: `lem`, `propext`, and `Quot.sound`. Now I'm trying to verify that D2 is truly the only field requiring `lem` by examining the individual components, though the aggregate audit makes this tricky to pin down precisely.

I can check by grepping the file to confirm `lem` appears only once, inside the `sober` proof, which would substantiate the claim. Actually, looking more carefully, all three axioms come from `sober` itself, so my comment about the classical frontier is consistent with that. I should verify the constructive parts by printing the axioms of `sierpTop` and the helper theorems to confirm they're empty, then maybe add those `#print axioms` statements to the code to demonstrate this.

Confirmed: the topology and helpers depend on no axioms at all, and `lem` appears only inside `sober`. Let me make that contrast machine-checked in the file rather than just asserted.

I've verified that `lem` only appears within `sober` and the topology with its helpers requires no axioms. Now I'll make this verifiable by adding a print statement to display the axioms of the topology layer before showing the full witness, so the axiom-free nature is machine-checked rather

than just claimed in a comment. Time to update the file's ending.

Yes — `instance` is only for `class` declarations. Since `DInfinityFoundations` is a `structure`, a witness is just an ordinary value: `def` if you want to name and reuse it, `example` if you only want to check that the axioms are satisfiable and throw the value away. I used `def` so it can be referenced and audited, and noted the `example` alternative in the comment.

For the type, I skipped the one-point space (every field holds vacuously, so it teaches nothing) in favor of **Sierpiński space** on `Bool` — `false` as $\perp$, `true` as $\top$, opens being the up-sets, which is exactly the condition $U \perp \rightarrow U \top$. It's the smallest non-degenerate Scott domain and it satisfies D1–D4 in full. It builds green with no `sorry`.

Two things fell out of it that are worth more than the code itself.

D1–D4 do not pin down D_∞ . A two-point space satisfies all four. The axioms characterize a *class* — pointed ω -algebraic domains — and reaching D_∞ specifically requires the tower and inverse limit as constructions on top, not consequences. That's stated in the header comment, and it reinforces the point Part I already makes about D being the "residue" rather than a definition of D_∞ .

The classical frontier is exactly D2. Compactness of the whole space has a pretty one-line proof here: any open covering $\perp$ is already everything, since opens are up-sets, so a single element of the cover suffices. That's constructive, as are T_0 , the basis conditions, and pointedness. Only sobriety needs `lem`, for the case split on whether `C  $\top$` holds. I made that machine-checked rather than asserted:

```
#print axioms sierpTop          -- does not depend on any axioms
#print axioms sierp_not_leq_top_bot -- does not depend on any axioms
#print axioms sierp             -- [lem, propext, Quot.sound]
```

The build echoes all three lines, so Layer O and the order facts come out axiom-free while the full witness reports `[lem, propext, Quot.sound]` — `propext` and `Quot.sound` arriving via `funext` on the closure equation, `lem` via that one case split. This matches what Part I of the paper says about D2 being where classical reasoning enters.

There are also four small named lemmas (`sierp_up`, `sierp_isOpen_top`, `sierp_leq_top`, `sierp_not_leq_top_bot`) that carry the reusable content, plus an `example` showing the order is genuinely two-element: `$\perp \sqsubseteq \top$` holds and `$\top \sqsubseteq \perp$` fails, so the space is T_0 but not T_1 .

Want me to add a short subsection to `LRSODInCIC.md` walking through this witness? It would give Part III a concrete payoff, and the axiom audit is a natural close for the constructivity note.

The builds are passing. For the terminology question, I'm clarifying that `instance` applies specifically to classes, while structures use `def` (or `example` for anonymous/discarded ones). I chose the Sierpiński space on `Bool` as the example since it's the smallest non-degenerate Scott domain, whereas `Unit` would be too trivial with everything being vacuous. The verification confirms all four domain axioms hold, and the audit trail shows the dependencies on lemmas, `propext`, and the soundness of quotient types.

All paths go through the sobriety condition, and the topology is axiom-free. The pedagogical insight here is that D_1 - D_4 alone don't uniquely determine D_∞ —even a two-point domain satisfies them. I could offer to add a `Unit` example for comparison or mention it in the documentation if that would be helpful.

Turn 11

Lars

Yes please

Composer 2.5 Fast

I'm deciding where to add a walkthrough of the Sierpiński witness in the [LRSODInCIC.md](#) document. The logical placement seems to be after the Layer D section, though I should consider whether it fits better after the Master correspondence section instead, depending on how it connects to the overall flow of Part III.

I'm leaning toward placing it right after Layer D as a concrete instantiation of those concepts, then having the constructivity note reference it. The subsection would cover terminology around instances versus definitions, the specific space structure with `Bool` and its opens, a table showing how each axiom verifies in this domain, key lessons about what the axioms do and don't determine, and a compactness argument. I should check the current section boundaries to place this accurately.

I need to update the constructivity note to reference the witness and check if the Master correspondence table should mention it too. I should also review whether the Vocabulary section's description of `instance` needs adjusting for consistency, then add a line to the constructivity note that points to the measured audit.

Added as “**A worked witness: the Sierpiński domain**”, placed right after Layer D so it reads as that layer's payoff, before the master correspondence table.

The subsection opens with the keyword point, since that's where the earlier vocabulary section earns its keep: `instance` registers a value for instance search and belongs to `class`; a `structure` witness is an ordinary value introduced with `def` or `example`. Stated plainly that there's no third mechanism — `instance` isn't the `structure` version of `def`, it's the `class` version.

Then the space itself (`Bool` , opens as up-sets, i.e. $U \perp \rightarrow U \top$), the `sierpTop` code, and a table showing how each of O1–O3 and D1–D4 gets discharged. The compactness argument gets called out because it's the prettiest part and shows D4 doing real work: any open covering $\perp$ is already the whole space, so one member of any cover suffices and the subcover is indexed by `Fin 1`.

It closes on the two conclusions. First, D1–D4 don't pin down D_∞ — a two-point space satisfies all four, so the axioms characterize a class of pointed ω -algebraic domains and the tower plus inverse limit sit on top of them rather than following from them. This makes Part I's "residue" framing concrete as a counterexample. Second, the classical frontier is exactly D2, with the three `#print axioms` lines quoted so the reader sees it measured rather than claimed.

I also appended a sentence to the constructivity note tying the table to that measurement: everything except sobriety is axiom-free, the completed witness reports `[lem, propext, Quot.sound]`, and `Classical.choice` appears nowhere.

The paper is 572 lines, the Lean file 324, and `lake build` is green with the audit lines echoing in the build output.